Matplotlib & Seaborn

Data Visualization in Python

Data Visualization in Exploratory Data Analysis (EDA)

- EDA is the first step in analyzing any dataset.
- It helps you understand the structure, relationships, and patterns in the data.
- It includes: Descriptive statistics (mean, median, std. deviation, etc.)Data visualization (graphs, plots, charts)

Role of Data Visualization in EDA

Visualization makes it easy to:

- Detect **patterns and trends**
- Spot **outliers/anomalies**
- Understand **distributions**
- Compare variables (**correlations & relationships**)

Common Visualization Techniques for EDA

1. Univariate Analysis (Single Variable)

- **Histogram** → Distribution of a numeric variable
- **Boxplot** → Spread, quartiles, outliers
- **Bar chart** → Frequency of categorical data

2. Bivariate Analysis (Two Variables)

- **Scatter Plot** → Relationship between two numerical variables
- **Line Plot** → Trends over time
- **Grouped Barplot** → Compare categories

3. Multivariate Analysis (Multiple Variables)

- **Pairplot (Seaborn)** → Relationship between multiple variables
- **Heatmap** → Correlation matrix
- **Stacked Barplot / Multiple Line Plots** → Compare across

Key Insights You Can Get

- **Distribution** → Is the data normal, skewed, or has outliers?
- **Relationships** → Are two variables correlated? (e.g., bill vs tip)
- **Categorical Comparisons** → Which group spends more?
- **Trends** → How values change with time or category.

Matplotlib –Introduction

- **Matplotlib** is a powerful **data visualization** library in Python.
- It helps create **2D plots, charts, and figures.**
- Works seamlessly with **NumPy** and **Pandas.**
- The main module:
- import matplotlib.pyplot as plt

Basic Plot

```
import matplotlib.pyplot as plt
import numpy as np
# Data
x = np.array([1, 2, 3, 4, 5])
y = x**2
# Simple Line Plot
plt.plot(x, y)
plt.title("Basic Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Multiple Lines in One Plot

- `x = np.linspace(0, 10, 100)`
- `plt.plot(x, np.sin(x), label="Sine", color="blue")`
- `plt.plot(x, np.cos(x), label="Cosine", color="red")`
- `plt.title("Sine and Cosine Waves")`
- `plt.xlabel("X values")`
- `plt.ylabel("Y values")`
- `plt.legend()`
- `plt.grid(True)`
- `plt.show()`

Markers and Line Styles

- `x = [1,2,3,4,5]`
- `y = [2,4,6,8,10]`
- `plt.plot(x, y, 'o--g', label="Dashed Green Line with Circles")`
- `# 'o' = marker, '--' = dashed line, 'g' = green`

- `plt.title("Markers & Styles Example")`
- `plt.legend()`
- `plt.show()`

Bar Chart

- `subjects = ['Math', 'Science', 'English', 'History']`
- `marks = [90, 85, 78, 88]`

- `plt.bar(subjects, marks, color='orange')`
- `plt.title("Bar Chart Example")`
- `plt.xlabel("Subjects")`
- `plt.ylabel("Marks")`
- `plt.show()`

Horizontal Bar Chart

- `plt.barh(subjects, marks, color='purple')`
- `plt.title("Horizontal Bar Chart")`
- `plt.show()`

Scatter Plot

- `x = np.random.rand(50)    # random values`
- `y = np.random.rand(50)`

- `plt.scatter(x, y, color="red", marker="*")`
- `plt.title("Scatter Plot Example")`
- `plt.show()`

Histogram

- `data = np.random.randn(1000)  # normally distributed data`

- `plt.hist(data, bins=30, color='skyblue', edgecolor='black')`
- `plt.title("Histogram Example")`
- `plt.xlabel("Values")`
- `plt.ylabel("Frequency")`
- `plt.show()`

Pie Chart

- `sizes = [30, 20, 25, 25]`
- `labels = ['Apples', 'Bananas', 'Cherries', 'Dates']`
- `colors = ['red', 'yellow', 'pink', 'brown']`

- `plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%')`
- `plt.title("Pie Chart Example")`
- `plt.show()`

Subplots (Multiple Graphs)

- `x = np.linspace(0, 10, 100)`
- `plt.subplot(2,2,1)   # 2 rows, 2 cols, position 1`
- `plt.plot(x, np.sin(x))`
- `plt.title("Sine")`

- `plt.subplot(2,2,2)`
- `plt.plot(x, np.cos(x))`
- `plt.title("Cosine")`

- `plt.subplot(2,2,3)`
- `plt.plot(x, np.tan(x))`
- `plt.title("Tangent")`

- `plt.subplot(2,2,4)`
- `plt.plot(x, np.exp(-x))`
- `plt.title("Exponential Decay")`

- `plt.tight_layout()`
- `plt.show()`

Customizing Plots

- `x = np.linspace(0, 5, 100)`
- `y = x**2`

- `plt.plot(x, y, color="blue", linewidth=2, linestyle="--", marker="o", markersize=6)`

- `plt.title("Customized Plot", fontsize=15, color="red")`
- `plt.xlabel("X axis", fontsize=12)`
- `plt.ylabel("Y axis", fontsize=12)`
- `plt.grid(True, linestyle=":")`
- `plt.show()`

Integration with Pandas

```
import pandas as pd

data = {
    "Year": [2018, 2019, 2020, 2021, 2022],
    "Sales": [250, 300, 400, 350, 500]
}

df = pd.DataFrame(data)

plt.plot(df["Year"], df["Sales"], marker="o")
plt.title("Company Sales Over Years")
plt.xlabel("Year")
plt.ylabel("Sales")
plt.show()
```

Seaborn

- It is built on top of **Matplotlib**, but provides a **higher-level API** with attractive default styles and built-in dataset support.
- Seaborn = Statistical data visualization library in Python.
- Built on **Matplotlib** and integrated with **Pandas**.
- Provides **beautiful plots with minimal code.**
- Import-------import seaborn as sns
- import matplotlib.pyplot as plt
- Seaborn comes with **built-in datasets** (like tips, iris, penguins).

Basic Example

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load example dataset
tips = sns.load_dataset("tips")

# Scatter plot
sns.scatterplot(x="total_bill", y="tip", data=tips)
plt.title("Scatter Plot of Tips Dataset")
plt.show()
```

Distribution Plot

- **Histogram + KDE (Kernel Density Estimation)**
- `sns.histplot(tips["total_bill"], bins=20, kde=True, color="skyblue")`
- `plt.title("Distribution of Total Bill")`
- `plt.show()`

Box Plot (for Outliers)

- `sns.boxplot(x="day", y="total_bill", data=tips, palette="Set2")`
- `plt.title("Boxplot: Total Bill by Day")`
- `plt.show()`

Violin Plot (Shape of Distribution)

- `sns.violinplot(x="day", y="total_bill", data=tips, palette="muted")`
- `plt.title("Violin Plot Example")`
- `plt.show()`

Bar Plot (Categorical Data)

- `sns.barplot(x="day", y="total_bill", data=tips, estimator=sum, ci=None, color="orange")`
- `plt.title("Total Bill by Day (Sum)")`
- `plt.show()`

Count Plot (Counts Categories)

- `sns.countplot(x="day", data=tips, palette="coolwarm")`
- `plt.title("Count of Customers by Day")`
- `plt.show()`

Pair Plot (Multi-variable Relationships)

- `sns.pairplot(tips, hue="sex")`
- `plt.suptitle("Pair Plot Example", y=1.02)`
- `plt.show()`

Heatmap (Correlation Matrix)

- `corr = tips.corr(numeric_only=True)`
- `sns.heatmap(corr, annot=True, cmap="coolwarm", linewidths=0.5)`
- `plt.title("Correlation Heatmap")`
- `plt.show()`

Regression Plot (Trend Line)

- `sns.regplot(x="total_bill", y="tip", data=tips, color="green")`
- `plt.title("Regression Line Example")`
- `plt.show()`

Style Customization

- `sns.set_style("darkgrid")  # styles: darkgrid, whitegrid, dark, white, ticks`
- `sns.set_palette("pastel")`

- `sns.scatterplot(x="total_bill", y="tip", data=tips, hue="day", style="time")`
- `plt.title("Customized Seaborn Plot")`
- `plt.show()`

Seaborn with Pandas DataFrame

```
import pandas as pd

data = {
    "Year": [2018, 2019, 2020, 2021, 2022],
    "Sales": [250, 300, 400, 350, 500],
    "Profit": [50, 70, 90, 80, 120]
}

df = pd.DataFrame(data)

sns.lineplot(x="Year", y="Sales", data=df, marker="o", label="Sales")
sns.lineplot(x="Year", y="Profit", data=df, marker="s", label="Profit")

plt.title("Sales & Profit over Years")
plt.legend()
plt.show()
```

Summary

- **Seaborn** makes plots easier & prettier compared to Matplotlib.
- Important plots:
  - scatterplot, histplot, boxplot, violinplot, barplot, countplot
  - pairplot, heatmap, regplot, lineplot
- Integrated with **Pandas DataFrames.**